Introdução a Aplicações Web Utilizando Banco de Dados MySql

Conteudista: Prof. Me. Alexander Gobbato Paulino Albuquerque
Revisão Textual: Prof.ª Dra. Selma Aparecida Cesarin

Objetivo da Unidade:

- Abordar os conceitos de JSP, JSTL, *Servlet* e padrão MVC para que, possamos criar uma aplicação completa utilizando Banco de Dados.

Material Teórico

Material Complementar

Referências

Material Teórico

Introdução

Olá, alunos(as)

Bem, as Unidades anteriores foram bastante produtivas, porém, até agora, não vimos como armazenar as informações em Banco.

Então, chegou o momento de juntarmos todos os estudos e acrescentar um local de armazenamento de informações.

Iremos utilizar o Banco de Dados relacional.

Mas o que significa relacional?

Um Banco de Dados relacional é um tipo de Banco no qual os dados são armazenados e existe um relacionamento entre esses dados armazenados.

Existem vários Bancos de Dados relacionais, mas, para o nosso estudo, iremos utilizar o *MySql*.

Site

MySql

Clique no botão para conferir o conteúdo.

ACESSE

Vamos relembrar rapidamente alguns pontos sobre os comandos de Banco:

- A linguagem *SQL* (*Structured Query Language*) é comum para os *SGBDR*;
- *SQLServer*, *Access*, *Oracle*, *MySQL*, *PostGree* etc.;
- Linguagem para manipulação de Bancos de Dados para inserção, remoção, pesquisa e alteração de dados.

Comandos *SQL*

- ***INSERT*:** insere um registro na tabela do Banco de Dados.

  Sintaxe: INSERT INTO <tabela> (<campos>) VALUES (<valores>)

- ***SELECT*:** pesquisa determinados registros em uma tabela.

Sintaxe: SELECT <campos> FROM <tabela> WHERE <condição>

- ***DELETE*:** remove registros de uma tabela;

  Sintaxe: DELETE FROM <tabela> WHERE <condição>

- ***UPDATE*:** atualiza registros em uma tabela;

  Sintaxe: UPDATE <tabela> SET <campos> = <valores> WHERE <condição>

Continuando ainda um pouco mais sobre Banco, precisamos de uma ferramenta para utilizar e, em nossos estudos, iremos utilizar o *Workbench*.

Site

MySQL *Workbench* 8.0.30

Clique no botão para conferir o conteúdo.

ACESSE

Feitas as devidas explicações, vamos colocar a mão na massa, e a primeira coisa que precisamos fazer é instalar o MySQL.



Figura 1 – Tela inicial *Workbench*
Fonte: Reprodução

Feitas as devidas explicações, vamos colocar a mão na massa, e a primeira coisa que precisamos fazer é instalar o MySQL.

Sites

MySQL Community Server 8.0.30
Clique no botão para conferir o conteúdo.

ACESSE

2.4 Installing MySQL on macOS
Clique no botão para conferir o conteúdo.

Ao acessar a página de *download*, selecione a versão do sistema operacional da sua máquina:

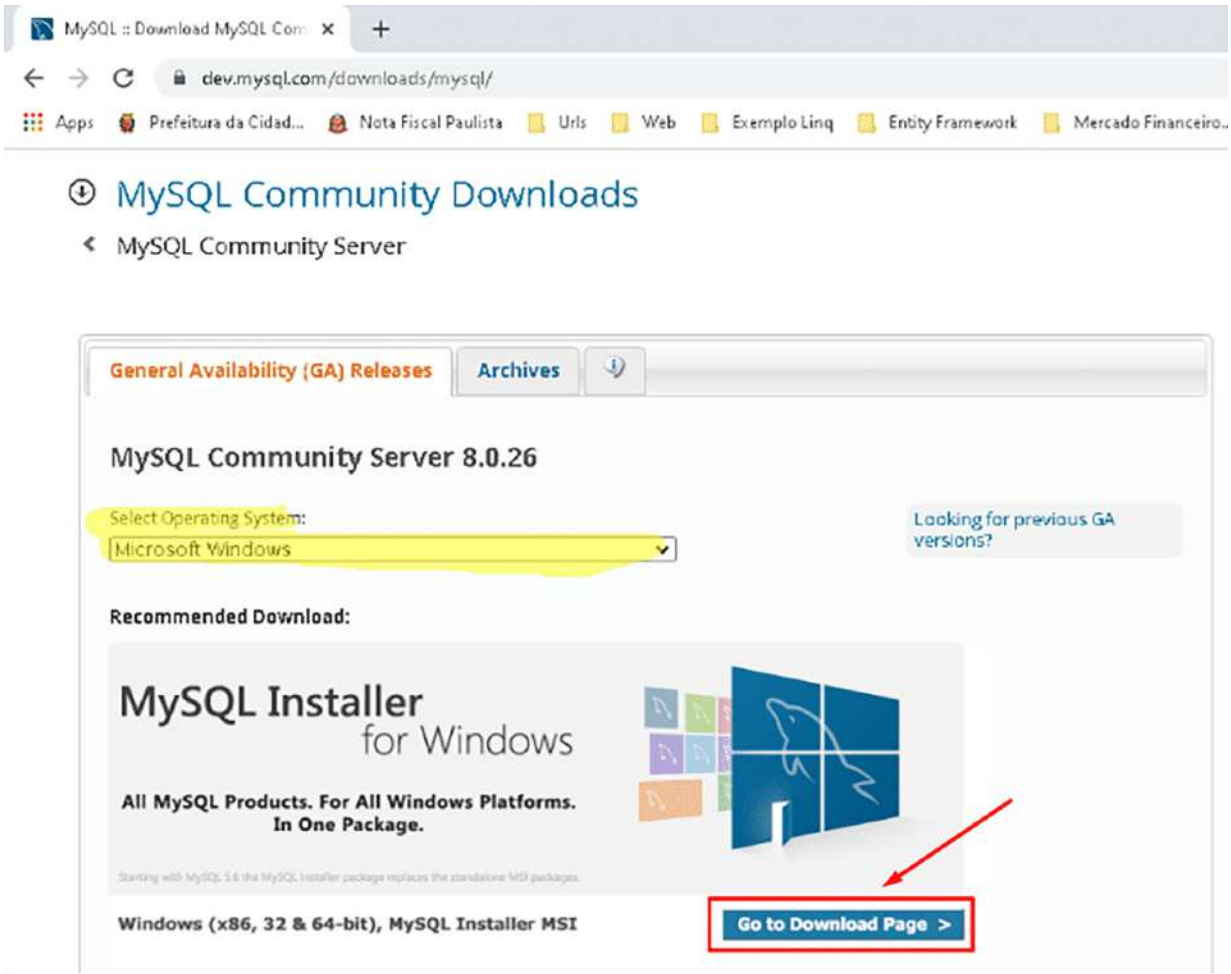


Figura 2 – Seleção do item para *download*
Fonte: Reprodução

Baixe o arquivo:

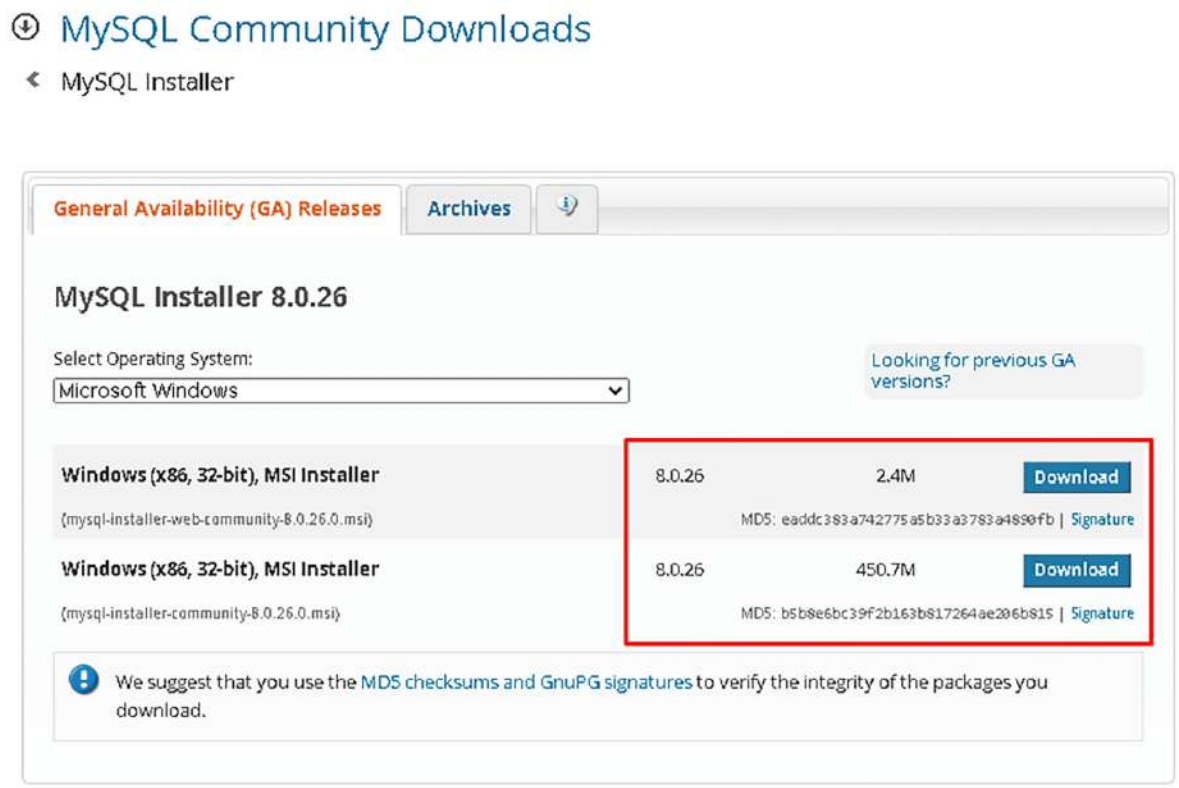


Figura 3 – Tipos de arquivo para instalação
Fonte: Reprodução

Não é necessário se cadastrar ou preencher uma lista de informações para ter acesso ao arquivo de instalação:

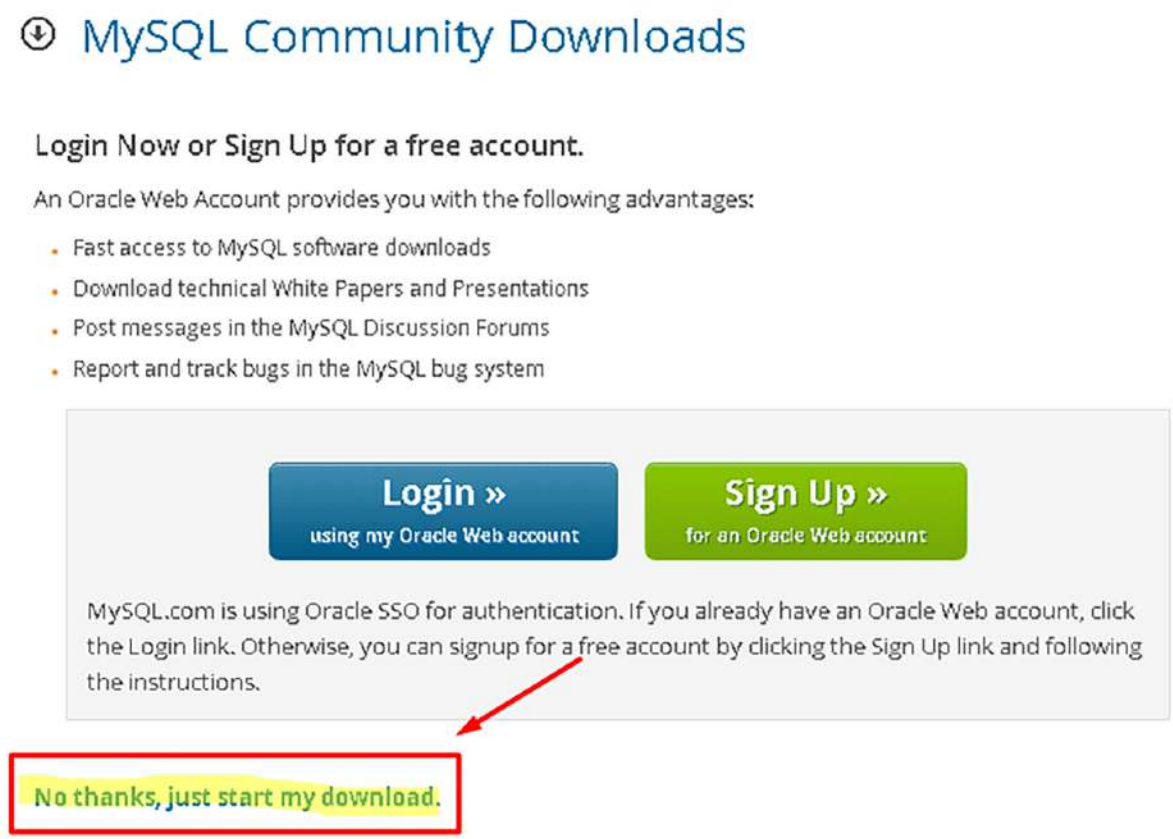


Figura 4 – *Link* para *download* sem a necessidade de *login*
Fonte: Reprodução

Após a instalação, utilizaremos o *Workbench*, e todos os *scripts* para esse aprendizado deverão ser executados nessa ferramenta:

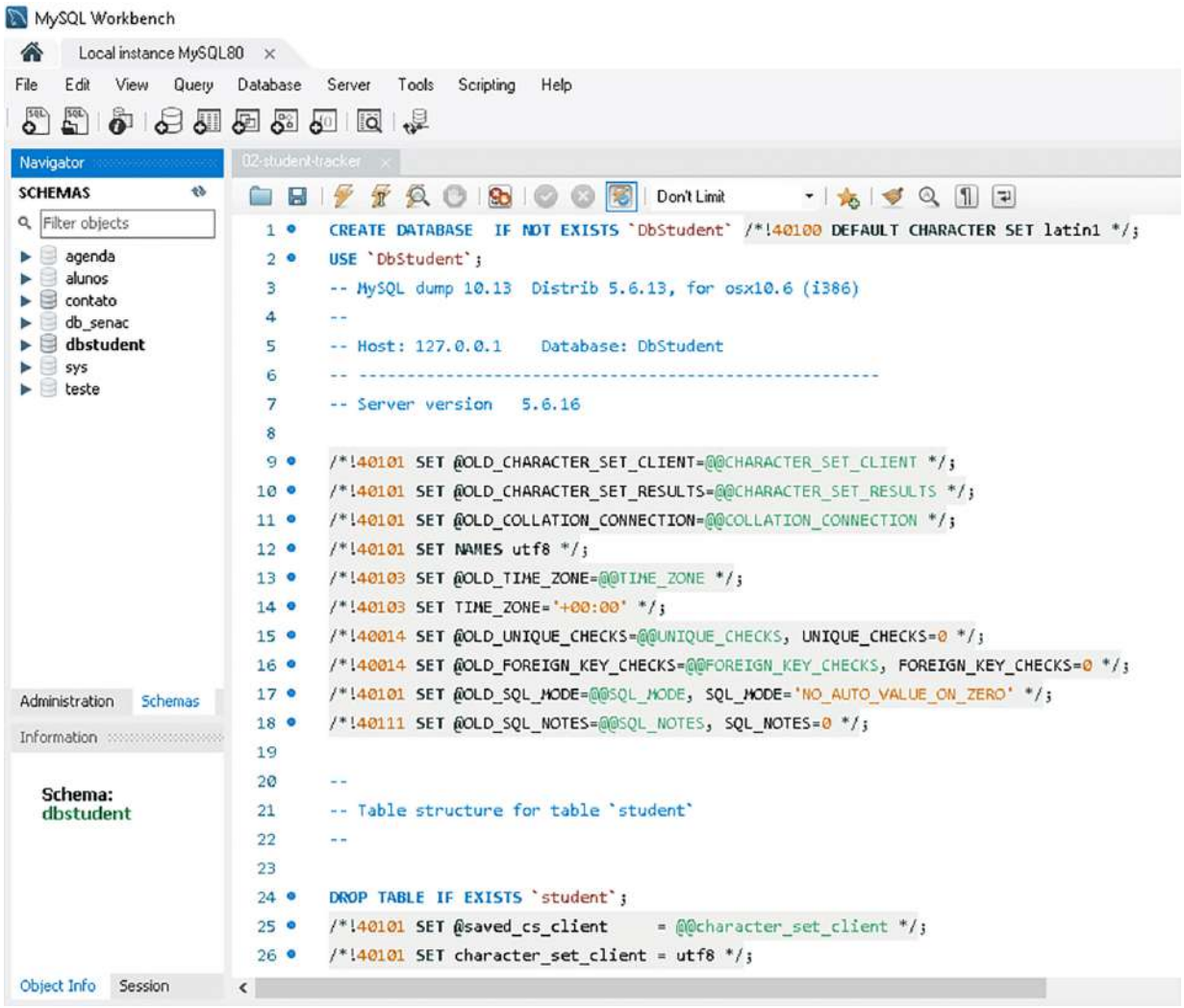


Figura 5 – *Script* para criação do banco
Fonte: Reprodução

Nesta Unidade, iremos aplicar o conceito de MVC estudado, ou seja, ao final, montaremos a seguinte estrutura:

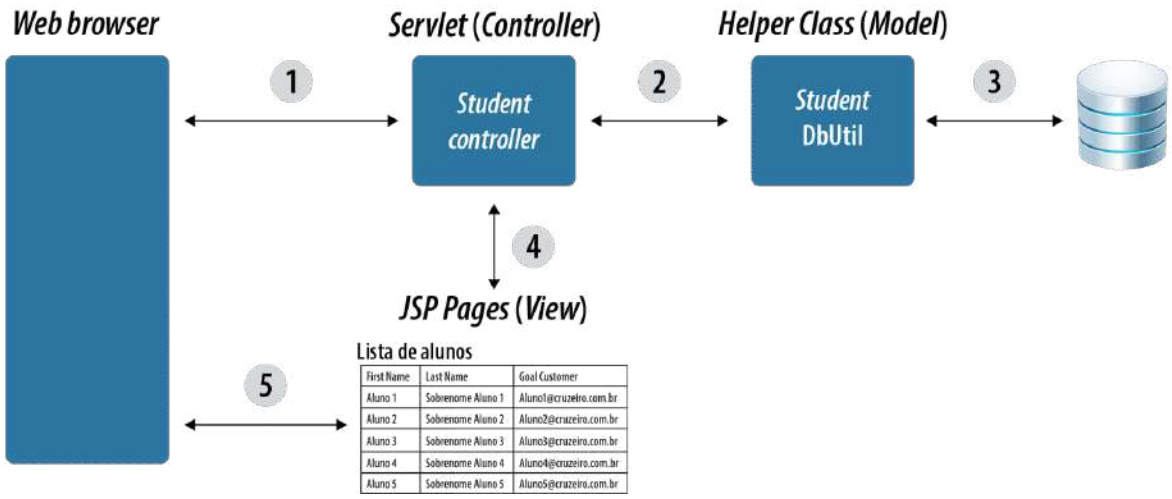


Figura 6 – Etapas para desenvolvimento
Fonte: Adaptada de Freepik

Então, vamos lá! O que iremos fazer a seguir é:

- Configurar o ambiente de Banco de Dados em desenvolvimento;
- Criar lista de estudantes;
- Adicionar um novo estudante;
- Alterar um estudante;
- Excluir um estudante.

Ao criar o projeto no *Eclipse*, iremos configurar a conexão ao Banco de Dados.

Lembre-se de que cada um pode ter criado uma senha na instalação. Nesta Unidade, foi criada uma senha padrão, sem se preocupar com a complexidade e a importância que a senha deve ter.

Lembre-se, também, de baixar todos as bibliotecas necessárias, a saber:

- Mysql-connector-java-8.0.13.jar;
- Taglibs-standard-compat-1.2.5.jar;
- Taglibs-standard-impl-1.2.5.jar;
- Taglibs-standard-jstlel-1.2.5.jar;
- Taglibs-standard-spec-1.2.5.jar.

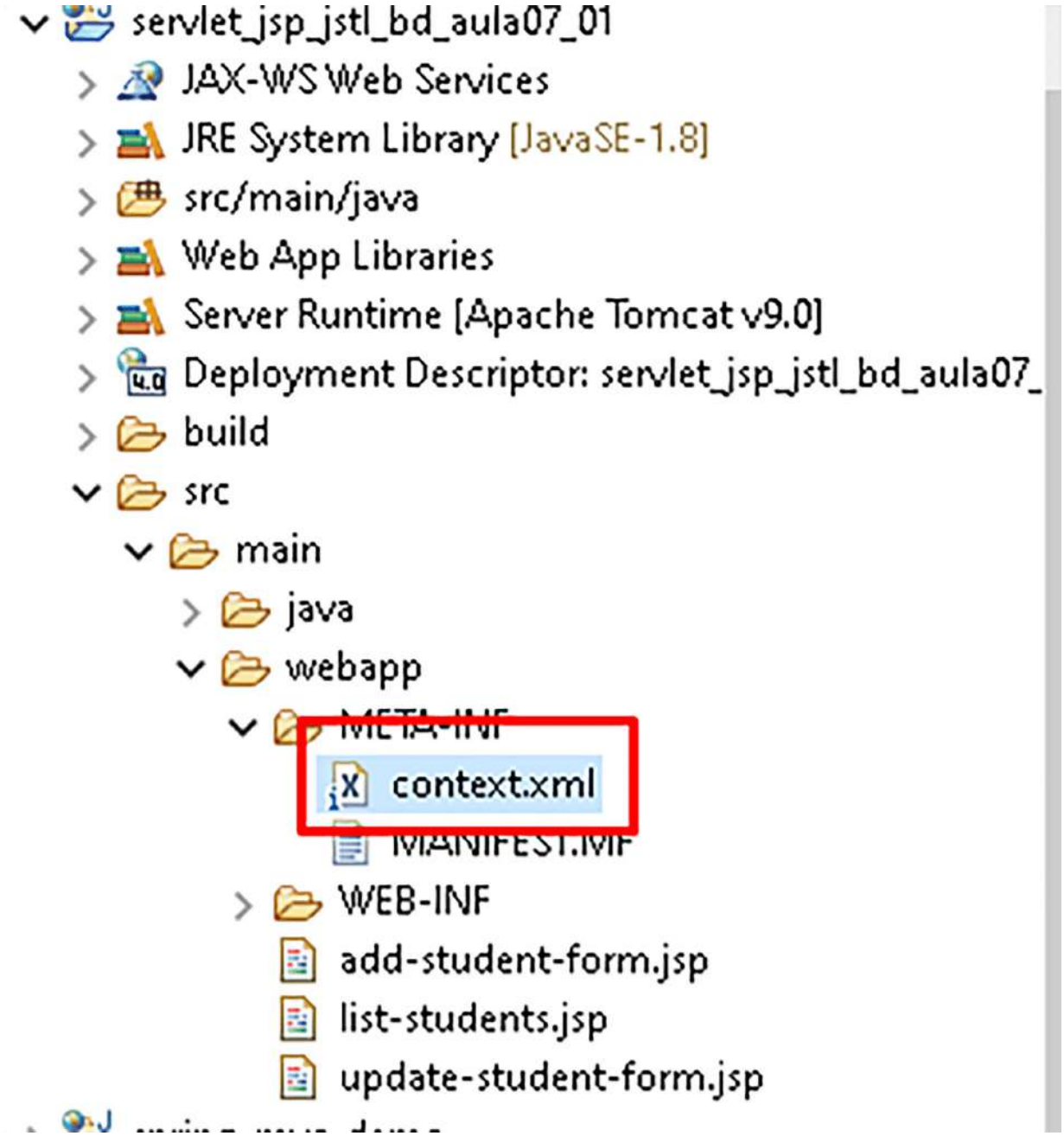


Figura 7 – Criando arquivo contexto.xml
Fonte: Reprodução

Definir a conexão no context.xml – META-INF/context.xml:



Figura 8 – Configurando o ambiente
Fonte: Reprodução

Vamos à explicação de cada item criado no *Resource*:

- *Name* = irá funcionar como um *alias* para futura chamadas;
- *Auth* = o *Tomcat* irá utilizar para se autenticar;
- *Type* = é a conexão genérica para a *pool*;
- *MaxActive* = Quantidade de conexões simultânea;
- *MaxIdle* = quantidade de usuários autenticados;
- *MaxWait* = tempo de espera de resposta do usuário para se conectar;
- *Username* e *password* = informações do Banco de Dados que iremos utilizar;
- *DriveClassName* = driver de conexão de um Banco específico;
- Url = caminho de acesso do Banco de Dados físico.

Criando o Objeto *Student*

Criar o objeto Student.java: objeto que irá representar a tabela do Banco de Dados. Como nosso projeto será no padrão MVC, essa classe será utilizada como model, ou seja, os mesmos elementos que existem na Tabela:

```java
package com.cruzeiro.jdbc;

public class Student {

    private int id;
    private String firstName;
    private String lastName;
    private String email;

    public Student() {

    public Student(String firstName, String lastName, String email) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
    }

    public Student(int id, String firstName, String lastName, String email) {
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
    }

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}
```

Figura 9 – Desenvolvendo a classe *Student*

Fonte: Reprodução

A próxima etapa é criarmos uma classe que terá alguns métodos para listar, inserir, alterar e excluir.

Classe *StudentDbUtil*

Na classe ***StudentDbUtil***, o atributo ***dataSource*** faz referência ao **xml** que foi criado no context.xml, que contém todos os elementos necessários para a conexão.

*StudentDbUtil.java

```
1  package com.cruzeiro.jdbc;
2
3  import java.sql.Connection;
12
13 public class StudentDbUtil {
14
15     public StudentDbUtil() {
16         // TODO Auto-generated constructor stub
17     }
18
19     private DataSource dataSource;
20
21     public StudentDbUtil(DataSource datasource) {
22         this.dataSource = datasource;
23     }
24
25 }
```

Figura 10 – Criando a classe StudentDbUtil
Fonte: Reprodução

```
private void close(Connection con, Statement stmt, ResultSet rs) {
    // TODO Auto-generated method stub

    try {
        if(rs!=null) {
            rs.close();
        }

        if(stmt != null) {
            stmt.close();
        }

        if(con!=null) {
            con.close();
        }

    }catch (Exception e) {
        // TODO: handle exception
        e.printStackTrace();
    }

}
```

Figura 11 – Criando os métodos
Fonte: Reprodução

O próximo passo é criar o método que irá realizar uma pesquisa no Banco e armazenar no objeto.

Observe que, no método chamado ***getStudents***, é criado um atributo do tipo ***ArrayList***, que pode armazenar objetos do tipo ***Student***.

A instrução **SELECT * FROM STUDENT** irá pesquisar todos os registros. Para cada registro, será gerado um objeto que será armazenado no objeto ***students***.

Nesse método, é necessário utilizar alguns objetos como ***Connection***, ***Statement*** e ***ResultSet***.

Veja o passo 1: está gerando o objeto *con*, para que possamos, por meio do ***createStatement***, executar a instrução SQL (Passo 2)

Com o retorno do Banco, realizar um laço, utilizando o comando **while(rs.next())**, ou seja, percorrer todos os resultados que foram armazenados no objeto ***rs***.

```
public List<Student> getStudents() throws Exception{

    List<Student> students = new ArrayList<>();

    Connection con = null;
    Statement stmt = null;
    ResultSet rs = null;

    try {
        //passo 1: criar a conexao
        con = datasource.getConnection();

        //passo 2: criar o comando sql
        String sql = "SELECT * FROM STUDENT ORDER BY LAST_NAME";
        stmt = con.createStatement();

        //passo 3: executar o comando
        rs = stmt.executeQuery(sql);

        //passo 4 : processar o resultado
        while(rs.next()) {
            //recuperar a informação que foi colocada no resultset
            int id = rs.getInt("id");
            String firstName = rs.getString("first_name");
            String lastName = rs.getString("last_name");
            String email = rs.getString("email");

            //popular o objeto student
            Student tempStudent = new Student(id, firstName, lastName, email);

            //adicioanr a lista
            students.add(tempStudent);
        }

        return students;
    }finally {
        //passo 4: fechar a conexao
        close(con, stmt, rs);
    }

}
```

Figura 12 – Criando método para retornar todos os estudantes

Fonte: Reprodução

Assim, já temos o objeto de conexão, o método de pesquisa. Então, agora, vamos para a criação do *servlet*, no qual iremos programar a regra de negócio.

O *servlet* ***StudentControllerServlet*** irá controlar o envio e o processo das informações do estudante.

Nesse *servlet*, iremos fazer uma chamada/referência para o *context* que criamos anteriormente.

Para inicializar o *servlet*, existe um método chamado **init()** e, na sequência, criar o método que irá carregar os dados do Banco, setar o resultado em uma variável e enviar para o jsp.

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    // listar os estudantes utilizando o pattern MVC
    try {
        listStudents(request, response);
    } catch (Exception e) {
        throw new ServletException(e);
    }
}

private void listStudents(HttpServletRequest request, HttpServletResponse response)
    throws Exception{
    //recuperar estudantes da banco de dados
    List<Student> students = studentDbUtil.getStudents();

    //adicionar os dados do request
    request.setAttribute("STUDENTS_LIST", students);

    //enviar para a página JSP (view)
    RequestDispatcher dispatcher = request.getRequestDispatcher("/list-students.jsp");
    dispatcher.forward(request, response);

}
```

Figura 13 – Implementando a chamada na *servlet*
Fonte: Reprodução

Agora, vamos para a última parte do desenvolvimento, a criação do JSP. Nessa página, serão feitas as referências aos objetos criados no *controller*, para que o conteúdo seja criado dinamicamente.

Após a criação, lembre-se de importar os pacotes que iremos utilizar nesse arquivo **(page import)** e JSTL. Portanto, não podemos esquecer do **<%@ taglib**

uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>

Criando a Página *List-students*

É o jsp que irá carregar as informações que foram preenchidas no *servlet controller*:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
    <div id="wrapper">
        <div id="header">
            <h2>Lista de Alunos</h2>
        </div>
        <div id="container">
            <table border="1">
                <tr>
                    <th>First Name</th>
                    <th>Last Name</th>
                    <th>Gold Customer</th>
                </tr>
                <c:forEach var="tempStudent" items="${STUDENTS_LIST}">
                    <tr>
                        <td>${tempStudent.firstName}</td>
                        <td>${tempStudent.lastName}</td>
                        <td>${tempStudent.email}</td>
                    </tr>
                </c:forEach>

            </table>
        </div>
    </div>

</body>
</html>
```

Figura 14 – Implementando a chamada na página
Fonte: Reprodução

Nesta primeira parte, montamos um projeto no padrão MVC:

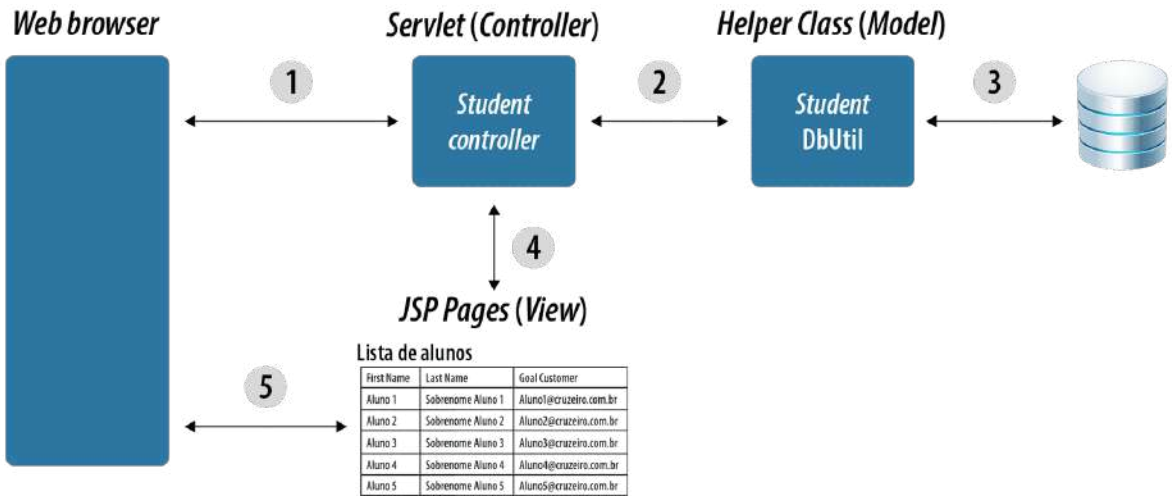


Figura 15 – Etapas para a conclusão das chamadas
Fonte: Adaptada de Freepik

Agora, iremos alterar o nosso programa para a criação do recurso de inserir aluno.

Então, devemos criar um formulário com os campos **First_Name**, **Last_Name** e **Email**.

Ajustar a lista estudantes para acrescentar um botão para adicionar o estudante:

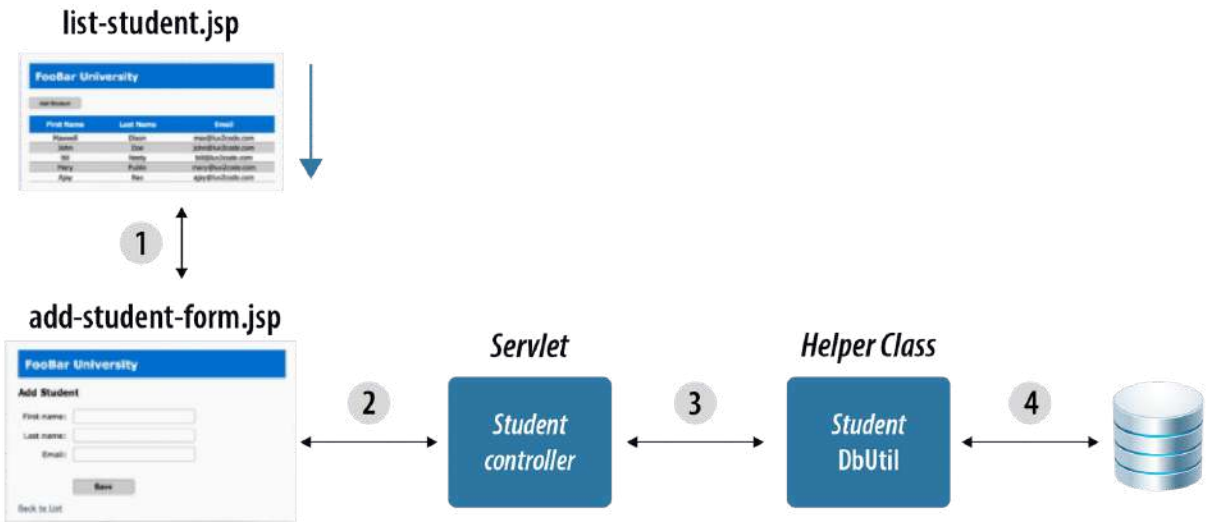


Figura 16 – Mapeando as chamadas para inclusão
Fonte: Adaptada de Freepik

Criação do Formulário

Observe um ponto importante no formulário: o ***action*** está direcionando para o *servlet* que criamos.

Na linha 21, foi criado um atributo ***hidden*** com o valor ***ADD***. Esse valor é de extrema importância porque, por meio dele, iremos verificar qual foi a ação escolhida pelo usuário:

add-student-form.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
    <div id="wrapper">
        <div id="header">
            <h2>Lista Alunos</h2>
        </div>
    </div>

    <div id="container">
        <h3>Adicionar Estudante</h3>

        <form action="StudentControllerServlet" method="GET">

            <input type="hidden" name="command" value="ADD" />

            <table>
                <tbody>
                    <tr>
                        <td><label>First name:</label></td>
                        <td><input type="text" name="firstName" /></td>
                    </tr>

                    <tr>
                        <td><label>Last name:</label></td>
                        <td><input type="text" name="lastName" /></td>
```

Figura 17 – Criando o form
Fonte: Reprodução

Essa verificação da ação ocorre na ***StudentControllerServlet***, na qual será direcionada para o recurso correto.

Para isso, foi criado um item de ***switch*** para que, dependendo do valor, o sistema seja direcionado para um código específico.

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    // listar os estudantes utilizando o pattern MVC
    try {
        //verificar o valor do parâmetro "command"
        String command = request.getParameter("command");

        //escolher a opção correta da operação
        if(command==null) {
            command = "LIST";
        }

        switch(command) {
        case "LIST":
            listStudents(request, response);
            break;
        case "ADD":
            addStudents(request,response);
            break;
        default:
            listStudents(request, response);

        }
    } catch (Exception e) {
        // TODO Auto-generated catch block
        throw new ServletException(e);
    }
}
```

Figura 18 – Implementando a *servlet*
Fonte: Reprodução

Veja como ficou o método ***addStudent*** no ***StudentControllerServlet:***

```
private void addStudents(HttpServletRequest request, HttpServletResponse response) throws Exception {
    //recuperar as informações enviadas
    String firstName = request.getParameter("firstName");
    String lastName = request.getParameter("lastName");
    String email = request.getParameter("email");

    //criar um novo objeto de estudante
    Student student = new Student(firstName, lastName, email);

    //gravar na base
    studentDbUtil.addStudent(student);

    //voltar para a lista de estudantes
    listStudents(request, response);


}
```

Figura 19 – Implementando a inclusão
Fonte: Reprodução

O ***request.getParameter*** é utilizado para “pegar” os valores que forem submetidos pelo formulário.

Com isso, podemos, então, armazenar os respectivos valores em variáveis e enviar para o recurso de gravação.

Site

Interface ServletRequest

“java.lang.String getParameter(java.lang.String name)

Clique no botão para conferir o conteúdo.

ACESSE

Retorna o valor de um parâmetro de solicitação como um String ou null se o parâmetro não existir. Os parâmetros de solicitação são informações extras enviadas com a solicitação. Para *servlets* HTTP, os parâmetros estão contidos na string de consulta ou nos dados de formulário postados.

Você só deve usar esse método quando tiver certeza de que o parâmetro tem apenas um valor. Se o parâmetro puder ter mais de um valor, use *getParameterValues*(java.lang.String).

- **Parametros:** Name – A String especificando o nome do parâmetro.

- **Devoluções:** a String representando o valor único do parâmetro.

Acrescentando o Método no StudentDbUtil para Inclusão

Vamos entender o que foi feito no código anterior.

```
public void addStudent(Student student)  throws Exception{
    //criar a instrução para gravar
    Connection con = null;
    PreparedStatement stmt = null;

    try {
        //configurar os valores dos estudantes
        con = dataSource.getConnection();

        String sql = "INSERT INTO STUDENT (FIRST_NAME, LAST_NAME, EMAIL) VALUES (?,?,?)";

        stmt = con.prepareStatement(sql);
        stmt.setString(1, student.getFirstName());
        stmt.setString(2, student.getLastName());
        stmt.setString(3, student.getEmail());

        //executar a instrução
        stmt.execute();

    }finally {
        //fehcar os objetos
        close(con, stmt, null);
    }
}
```

Figura 20 – Criando o método para gravação
Fonte: Reprodução

Vamos entender o que foi feito no código anterior.

Observe que foi criada uma variável de nome sql e atribuímos um texto que, ao ser enviado para o Banco de Dados, serão salvos na tabela os valores, mas o que seria o caractere "?" que estamos vendo na instrução?

Esse *caracter* "?" é uma espécie de coringa, ou seja, ao configurarmos os valores no objeto stmt:

```
stmt = con.prepareStatement(sql);
stmt.setString(1, student.getFirstName());
stmt.setString(2, student.getLastName());
stmt.setString(3, student.getEmail());
```

Figura 21
Fonte: Reprodução

Estamos buscando o valor que foi alimentado no objeto *student*, os numerais 1, 2 e 3 substituem o "?" na sequência em que foram criados na instrução *INSERT*.

Pronto!

Nosso método de gravação está finalizado.

Vamos adaptar, agora, o nosso projeto para realizar a alteração do estudante.

O processo é muito parecido com a inclusão.

Veja: vamos precisar criar um formulário e ajustar a lista de estudantes para acrescentar a opção de alterar:

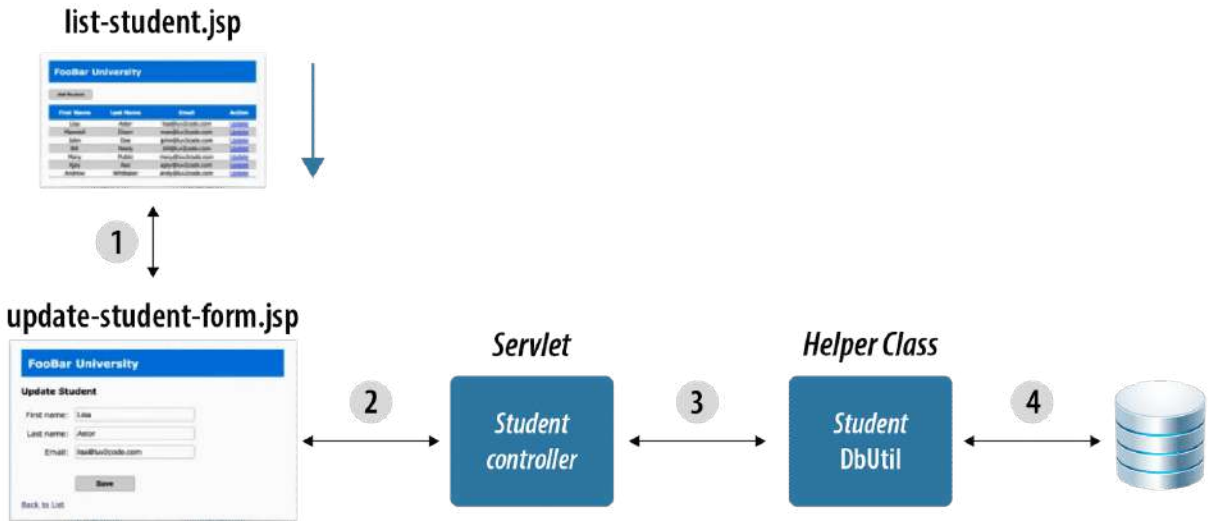


Figura 22 – Passo a passo para a alteração
Fonte: Adaptada de Freepik

Passo a passo:

1. Alterar o arquivo list-students.jsp – Devemos adicionar uma ação;
2. Criar um formulário para o alterar estudante;
3. Alterar o *StudentControllerServlet* – Devemos capturar o *request* de alteração do estudante;
4. Alterar o *StudentDbUtil* – Devemos criar um método para alterar o estudante.

Alterando a Lista do Estudante

Observe que, agora, estamos utilizando o comando c:url. Esse comando disponibiliza uma url que vai ser direcionada para o componente que está preenchido no atributo value.

```
<div id="wrapper">
    <div id="header">
        <h2>Lista de Alunos</h2>
    </div>
    <div id="content">
        <input type="button" value="Add Student" onclick="addstudent()" />
        <table border="1">
            <tr>
                <th>First Name</th>
                <th>Last Name</th>
                <th>Email</th>
                <th>Action</th>
            </tr>
            <c:forEach var="tempStudent" items="${STUDENTS_LIST}">

                <c:url var="tempLink" value="StudentControllerServlet">
                    <c:param name="command" value="LOAD"/>
                    <c:param name="studentId" value="${tempStudent.id}"/>
                </c:url>

                <tr>
                    <td>${tempStudent.firstName}</td>
                    <td>${tempStudent.lastName}</td>
                    <td>${tempStudent.email}</td>
                    <td><a href="${tempLink}">Update</a></td>
                </tr>
            </c:forEach>

        </table>
    </div>
</div>
```

Figura 23 – Modificando o *form*
Fonte: Reprodução

Observe que, agora, estamos utilizando o comando c:url. Esse comando disponibiliza uma url que vai ser direcionada para o componente que está preenchido no atributo *value*.

Acrescentamos, ainda, os parâmetros *command* com o valor *LOAD* e o parâmetro *studentId* com o valor do id do estudante, que será gerado com o retorno do Banco.

O atributo *name=command* é importante para que, quando o *servlet* receber a requisição, ele possa saber qual operação estamos trabalhando.

Veja o código seguinte com o tratamento do valor *LOAD*:

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    // listar os estudantes utilizando o pattern MVC
    try {
        //verificar o valor do parâmetro "command"
        String command = request.getParameter("command");

        //escolher a opção correta da operação
        if(command==null) {
            command = "LIST";
        }

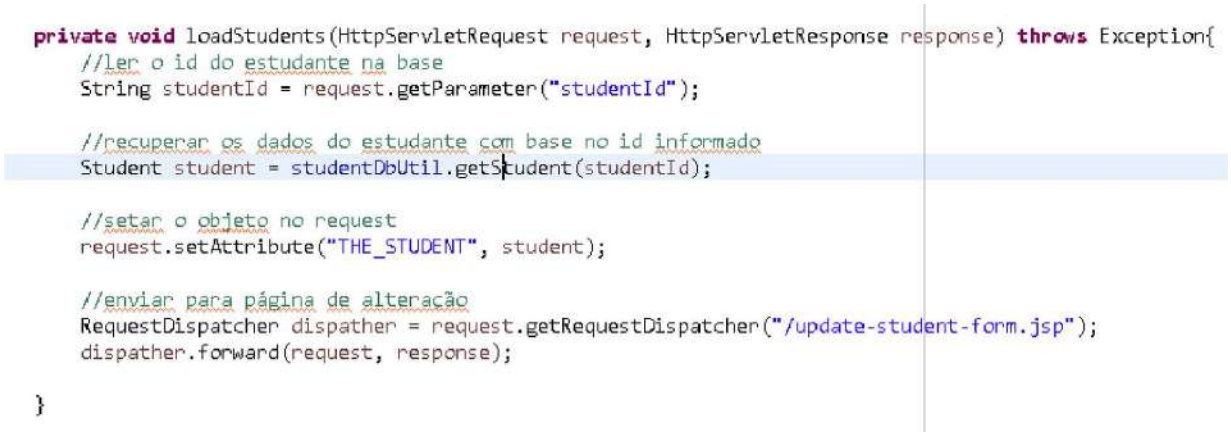
        switch(command) {
        case "LIST":
            listStudents(request, response);
            break;
        case "ADD":
            addStudents(request,response);
            break;
        case "LOAD":
            loadStudents(request,response);
            break;
        default:
            listStudents(request, response);

        }
    } catch (Exception e) {
        // TODO Auto-generated catch block
        throw new ServletException(e);
    }
}
```

Figura 24 – Ajustando a chamada para alteração
Fonte: Reprodução

Quando for enviado o valor *LOAD*, direcionaremos para a função *loadStudent*.

O *request.setAttribute* irá armazenar um valor que será utilizado na página JSP que o *RequestDispatcher* irá direcionar:



```
private void loadStudents(HttpServletRequest request, HttpServletResponse response) throws Exception{
    //ler o id do estudante na base
    String studentId = request.getParameter("studentId");

    //recuperar os dados do estudante com base no id informado
    Student student = studentDbUtil.getStudent(studentId);

    //setar o objeto no request
    request.setAttribute("THE_STUDENT", student);

    //enviar para página de alteração
    RequestDispatcher dispather = request.getRequestDispatcher("/update-student-form.jsp");
    dispather.forward(request, response);

}
```

Figura 25 – Criando o método para carregar as informações
Fonte: Reprodução

Site

Interface ServletRequest

“void setParameter(java.lang.String name,java.lang.Object o)

Clique no botão para conferir o conteúdo.

ACESSE

Armazena um atributo nessa solicitação. Os atributos são redefinidos entre as solicitações. Este método é mais frequentemente usado em conjunto com *RequestDispatcher*.

Os nomes dos atributos devem seguir as mesmas convenções dos nomes dos pacotes. Os nomes que começam com java.*, javax.*e com.sun.* são reservados para uso pela *Sun Microsystems*.

Se o objeto passado for nulo, o efeito será o mesmo que chamar removeAttribute(java.lang.String).

É avisado que quando a solicitação é despachada do *servlet*, reside em um aplicativo da *Web* diferente por *RequestDispatcher*. O objeto definido por esse método pode não ser recuperado corretamente no *servlet* chamador.

- Name – A String especificando o nome do atributo
  o – Objeto que será armazenado".

Site

Interface ServletRequest

"RequestDispatcher getRequestDispatcher(java.lang.String path)

Clique no botão para conferir o conteúdo.

ACESSE

Retorna um *RequestDispatcher* objeto que atua como um wrapper para o recurso localizado no caminho fornecido. Um objeto *RequestDispatcher* pode ser usado para encaminhar uma solicitação ao recurso ou para incluir o recurso em uma resposta. O recurso pode ser dinâmico ou estático.

O nome do caminho especificado pode ser relativo, embora não possa se estender para fora do contexto do *servlet* atual. Se o caminho começar com um "/", ele será interpretado como relativo à raiz de contexto atual. Esse método retorna null se o contêiner do *servlet* não puder retornar um arquivo *RequestDispatcher*.

- **Parâmetros:** *PATH* – O texto especificando do nome do caminho para o recurso. Se for relativo, deve ser relativo ao *servlet* atual;
- **Devoluções:** um *RequestDispatcher*objeto que atua como um wrapper para o recurso no caminho especificado ou null se o contêiner do *servlet* não puder retornar um *RequestDispatcher*.

Observe que o formulário para atender a requisição do *LOAD* será preenchido com os dados que foram encontrados no Banco de Dados.

Veja que os atributos hmtl *input type* foram acrescidos com o *value*.

Notem que o objeto THE_STUDENT é o que foi carregado no controller. Dessa forma, podemos enviar os valores de uma camada para outra:



Figura 26 – Ajustando o *form*
Fonte: Reprodução

Quando o usuário clicar no botão para salvar, ele será enviado para o *StudentControllerServlet*.

Agora, vejamos o código:

```
private void updateStudents(HttpServletRequest request, HttpServletResponse response) throws Exception {
    //ler a informação
    int id = Integer.parseInt(request.getParameter("studentId"));
    String firstName = request.getParameter("firstName");
    String lastName = request.getParameter("lastName");
    String email = request.getParameter("email");

    //criar um objeto de students
    Student student = new Student(id,firstName, lastName, email);

    //alterar os dados na base
    studentDbUtil.updateStudent(student);

    //voltar par alista de estudantes
    listStudents(request, response);

}
```

Figura 27 – Criando a chamada para alteração
Fonte: Reprodução

Nesse código, recuperamos os valores que foram enviados no *form*. Veja que estamos utilizando novamente o *request.getParameter*.

Na sequência, preenchemos o objeto *Student* e executamos o método *updateStudent* do *StudentDbUtil*:

```java
public void updateStudent(Student student) throws Exception {
    // criar a instrução para alterar
    Connection con = null;
    PreparedStatement stmt = null;

    try {
        // configurar os valores dos estudantes
        con = dataSource.getConnection();

        String sql = "UPDATE STUDENT SET FIRST_NAME = ?, LAST_NAME = ?, EMAIL = ? WHERE ID = ?";

        stmt = con.prepareStatement(sql);
        stmt.setString(1, student.getFirstName());
        stmt.setString(2, student.getLastName());
        stmt.setString(3, student.getEmail());
        stmt.setInt(4, student.getId());

        // executar a instrução
        stmt.execute();

    } finally {
        // fehcar os objetos
        close(con, stmt, null);
    }

}
```

Figura 28 – Criando o método para gravar no banco a alteração
Fonte: Reprodução

Para finalizarmos, é necessário somente fazer um ajuste para incluirmos a opção de exclusão:

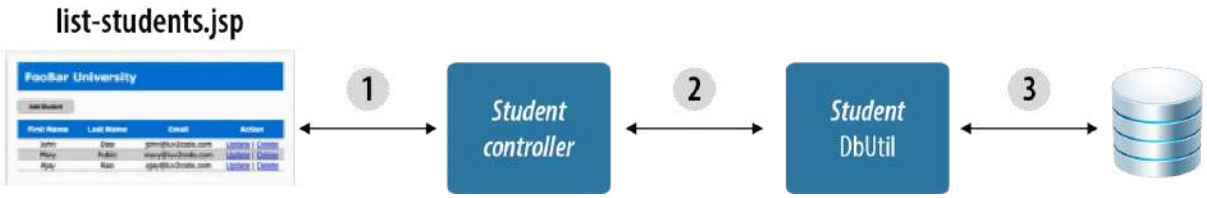


Figura 29 – Passo a passo para listar os estudantes
Fonte: Adaptada de Freepik

O que devemos fazer é:

1. Alterar o arquivo list-students.jsp – Devemos adicionar uma ação;
2. Alterar o StudentControllerServlet – Devemos capturar o *request* de exclusão do estudante;
3. Alterar o StudentDbUtil – Devemos criar um método para excluir o estudante.

Vamos Entender o Código

No arquivo .jsp, iremos incluir um item para que quando o usuário clique ele pergunte se quer realmente excluir o estudante:

```
<c:forEach var="tempStudent" items="${STUDENTS_LIST}">

    <c:url var="tempLink" value="StudentControllerServlet">
        <c:param name="command" value="LOAD"/>
        <c:param name="studentId" value="${tempStudent.id}"/>
    </c:url>

    <c:url var="tempLinkDelete" value="StudentControllerServlet">
        <c:param name="command" value="DELETE"/>
        <c:param name="studentId" value="${tempStudent.id}"/>
    </c:url>

    <tr>
        <td>${tempStudent.firstName}</td>
        <td>${tempStudent.lastName}</td>
        <td>${tempStudent.email}</td>
        <td>
            <a href="${tempLink}">Update</a>
            |
            <a href="${tempLinkDelete}" onclick="delstudent()">Delete</a>
        </td>
    </tr>
</c:forEach>
```

Figura 30 – Ajustando o *form*
Fonte: Reprodução

Devemos alterar o *StudentControllerServlet* para receber o valor da variável de excluir e, então, direcionar para o método correto, lembrando-se de que, como se trata de uma exclusão, precisamos saber qual estudante irá ser excluído.

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    // listar os estudantes utilizando o pattern MVC
    try {
        //verificar o valor do parâmetro "command"
        String command = request.getParameter("command");

        //escolher a opção correta da operação
        if(command==null) {
            command = "LIST";
        }

        switch(command) {
        case "LIST":
            listStudents(request, response);
            break;
        case "ADD":
            addStudents(request,response);
            break;
        case "LOAD":
            loadStudents(request,response);
            break;
        case "UPDATE":
            updateStudents(request,response);
            break;
        case "DELETE":
            deleteStudents(request,response);
            break;
        default:
            listStudents(request, response);

        }
    } catch (Exception e) {
        // TODO Auto-generated catch block
        throw new ServletException(e);
    }
}
```

Figura 31 – Acrescentando a chamada para exclusão
Fonte: Reprodução

No arquivo *StudentDbUtil*, iremos criar o método de exclusão:

```
public void deleteStudents(int id)throws Exception {
    // criar a instrução para excluir
    Connection con = null;
    PreparedStatement stmt = null;

    try {
        //concetar no banco
        con = dataSource.getConnection();

        //configurar  a instrução
        String sql = "DELETE FROM  STUDENT WHERE ID = ?";

        //preparar a instrução
        stmt = con.prepareStatement(sql);

        //configurar os parametros
        stmt.setInt(1, id);

        //executar a instrução
        stmt.execute();

    } finally {
        // fehcar os objetos
        close(con, stmt, null);
    }

}
```

Figura 32 – Criando a chamada para exclusão
Fonte: Reprodução

Pronto, acabamos de concluir um projeto que faz pesquisa, inclusão, alteração e exclusão: o famoso *CRUD*, *Create Read Update Delete*.

Só mais um detalhe: podemos configurar o arquivo web.xml da aplicação para quando digitarmos a url. Ele já executa uma página *default*.

Configurando o Arquivo web.xml

O arquivo web.xml fornece informações de configuração e de implementação para os componentes da *Web* que compreendem um aplicativo da *Web*.

A especificação *Java™ Servlet* define o arquivo do descritor de implementação web.xml em termos de um documento de esquema XML.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Programando com Asp.net mvc: Aprenda a Desenvolver Aplicações *Web* Utilizando a Arquitetura mvc

LOTAR, A. **Programando com asp.net mvc:** aprenda a desenvolver aplicações *web* utilizando a arquitetura mvc. São Paulo: Novatec, 2011.

Especificações Técnicas de *Software*

LIMA, A. S. **Especificações técnicas de *software***. São Paulo: Érica, 2012. 384p.

Java para *Web*: Desenvolvimento de Aplicações

ALVES, W. P. **Java para *Web*:** desenvolvimento de aplicações. São Paulo: Erica, 2015.

Desenvolvendo Aplicações *Web* com JSP, *Servelts*, *Javaserver Faces*, *Hibernate*, EJB 3 *Persistance* e *Ajax*

GONÇALVES, E. **Desenvolvendo aplicações *Web* com JSP, *Servelts*, *Javaserver Faces*, *Hibernate*, EJB 3 *Persistance* e *Ajax***. Rio de Janeiro: Ciência Moderna, 2007.

Referências

GETPARAMETER – INTERFACE SERVLETREQUEST. ***Oracle Help Center***, *s. d.* Disponível em: <https://docs.oracle.com/javaee/6/api/javax/servlet/ServletRequest.html#getParameter(java.lang.String)>. Acesso em 08/07/2022.

GETREQUESTDISPATCHER – INTERFACE SERVLETREQUEST. ***Oracle Help Center***, *s. d.* Disponível em: <https://docs.oracle.com/javaee/6/api/javax/servlet/ServletRequest.html#getRequestDispatcher(java.lang.String)>. Acesso em 08/07/2022.

SETATTRIBUTE – INTERFACE SERVLETREQUEST. ***Oracle Help Center***, *s. d.* Disponível em: <https://docs.oracle.com/javaee/6/api/javax/servlet/ServletRequest.html#setAttribute(java.lang.String,%20java.lang.Object)>. Acesso em 08/07/2022.